KICS The Korean Institute of Communications and Information Sciences

Available online at www.sciencedirect.com
ScienceDirect

ICT Express 10 (2024) 213–231

www.elsevier.com/locate/icte

Training-free neural architecture search: A review

Meng-Ting Wu, Chun-Wei Tsai*

Department of Computer Science and Engineering, National Sun Yat-sen University, Kaohsiung, Taiwan

Received 17 February 2023; received in revised form 17 June 2023; accepted 3 November 2023
Available online 10 November 2023

Abstract

The goal of neural architecture search (NAS) is to either downsize the neural architecture and model of a deep neural network (DNN), adjust a neural architecture to improve its end result, or even speed up the whole training process. Such improvements make it possible to generate or install the model of a DNN on a small device, such as a device of internet of things or wireless sensor network. Because most NAS algorithms are time-consuming, finding out a way to reduce their computation costs has now become a critical research issue. The training-free method (also called the zero-shot learning) provides an alternative way to estimate how good a neural architecture is more efficiently during the process of NAS by using a lightweight score function instead of a general training process to avoid incurring heavy costs. This paper starts with a brief discussion of DNN and NAS, followed by a brief review of both model-dependent and model-independent training-free score functions. A brief introduction to the search algorithms and benchmarks that were widely used in a training-free NAS will also be given in this paper. The changes, potential, open issues, and future trends of this research topic are then addressed in the end of this paper.

© 2023 The Author(s). Published by Elsevier B.V. on behalf of The Korean Institute of Communications and Information Sciences. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Keywords: Neural architecture search; Deep neural network; Training-free; Zero-shot; Internet of things

1. Introduction

The deep neural network (DNN) has been widely discussed from different perspectives of the basic idea, theory, algorithm, applications, and systems in several recent studies [1–3]. Since problems people want to solve become more diverse, various types of deep neural networks are introduced to generate suitable classification or forecasting models for different purposes [4]. Studies [5,6] pointed out that both increasing the number of hidden layers of a DNN and adding an appropriate combination of neurons and layers to a DNN may improve the accuracy of the model; however, they are not the only ways. In this sense, the "deep" in a deep neural network (DNN) implies that a larger and more complex network is used to gain the desired accuracy. A simple example demonstrating the expressiveness of a deeper and more complex neural network called LeNet, which is one of the precursor convolutional neural network (CNN) architectures and one of the most well-known DNNs, can be found in [5]. Different from a simple neural network (NN) that uses only fully connected layers as the hidden layers, LeNet [5] uses both convolutional and pooling layers. In [6], Alex et al. pointed out that a possible way to increase the accuracy of a model is to add additional convolutional and pooling layers. Using an appropriate combination of neurons, of course, is another way to improve the model accuracy [7].

Although a more complex, deeper, and larger DNN architecture can be invented to gain higher accuracy, it will also incur more computation costs. An intuitive solution is to use a cloud computing platform to accelerate the running time of training and inference of a DNN model. A growing number of tiny devices are miniaturized and low-power embedded, sensor and microcontroller unit (MCU) are used in emerging wireless sensor network (WSN) and internet of things (IoT) environments, cloud computing is no longer the panacea because offloading the computations of a DNN from end devices to a cloud will confront new dilemmas. Among them are the bandwidth limitations between end devices and the cloud, high network latency between end devices and the cloud, security and privacy of local data, and even more the energy efficiency issues [8–10]. That is why the computations of DNN on mobile and handheld devices have been shifted from cloud computing and mobile cloud computing (MCC), to local cloudlet, cloudlet, edge and fog, and then to mobile edge computing (MEC) [11]. Since these changes, several recent

* Corresponding author.
E-mail address: cwtsai@mail.cse.nsysu.edu.tw (C.-W. Tsai).
Peer review under responsibility of The Korean Institute of Communications and Information Sciences (KICS).

https://doi.org/10.1016/j.icte.2023.11.001
2405-9595/© 2023 The Author(s). Published by Elsevier B.V. on behalf of The Korean Institute of Communications and Information Sciences. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

studies [8,11] have indicated that letting the endpoint device perform its own learning and analytics independently as a component of an intelligent system can reduce the latency and bandwidth requirement as well as improve data security and privacy.

Existing tiny devices, however, have two major constraints [8]: (1) the clock speed of the processor of such devices is usually limited to 10–1000 MHz and (2) the storage and memory are typically quite small, especially in an MCU, which typically has less than 1 MB on-board flash memory with 1 MB SRAM [12,13]. Due to limitations of computing power, sizes of storage and memory, and even the energy, most tiny devices obviously cannot be equipped for complex DNN learning models. Several recent studies [9,14,15] have then presented various compression and acceleration technologies to reduce computation costs of a DNN and the DNN model size to meet the requirements of a sensor or MCU to make it possible to install DNN or its model on such tiny devices.

In addition to finding the way to put a DNN onto tiny devices, how to remove human involvement from the learning process of a DNN today has become a critical research issue. That is because the whole process of most learning methods nowadays still requires the involvement of humans, such as data cleaning, parameter adjustment, or model redesign. From this perspective, several recent studies [16–18] attempted to find ways to design self-adaptive learning methods to automatically generate suitable learning models to realize an intelligent system, called the automated machine learning (AutoML). Among them, the hyperparameter optimization (HPO) and neural architecture search (NAS) are two promising research topics in DNN. Apart from hyperparameter optimization [19] that optimizes only training-related parameters, the NAS [20,21] is a promising research topic because the basic idea of NAS is to use a search algorithm to search for a suitable neural architecture from a set of possible neural architectures, thus making it possible to find a more effective neural architecture to improve the accuracy or reduce the model size, which will in turn speed up the computation time of forecasting or classification process. The search algorithm used can be either a simple greedy algorithm, a reinforcement learning algorithm, or a metaheuristic algorithm. In most studies of NAS [20,21], a common approach to evaluating the quality of each neural architecture resulted from the search process is to train the neural architecture (e.g., by using a deep learning process to train the neural architecture to get the training error) to evaluate how good the neural architecture is.

Fig. 1 gives a simple example to explain how to use semi-supervised or unsupervised algorithms as the search algorithm to look for a suitable neural architecture (a solution s) from a set of possible neural architectures and how to use the training and validation processes of deep learning to get the training error as the objective value of a suitable neural architecture s for the search algorithm. Similar to most semi-supervised and unsupervised algorithms for solving optimization problems [22], they all contain the following operators: initialization, transition, evaluation, and determination. The *initialization* operator is used to generate the initial solution(s) by using a random process. In this example, each solution s can be regarded as a neural architecture of DNN. Some refinement or heuristic algorithms can also be used to generate the initial solution(s) if we have the domain knowledge of the problem or application in question. The *transition* operator plays the role of generating new candidate solution(s) by changing the current solution(s) or creating brand-new candidate solution(s). The *evaluation* operator is responsible for evaluating the objective or fitness value of each new candidate solution, which typically represents how good a candidate neural architecture trained by using the training data $\mathcal{D}_{\text{Train}}$ and estimated by using the validation data $\mathcal{D}_{\text{Valid}}$ is. Thus, the training error $\mathcal{E}_{\text{Train}}$ is widely used in estimating the quality of a candidate neural architecture. The determination operator plays the role of guiding the searches of the search algorithm during the convergence process by using the objective or fitness values of solution(s) obtained by the evaluation operator and other relevant information.

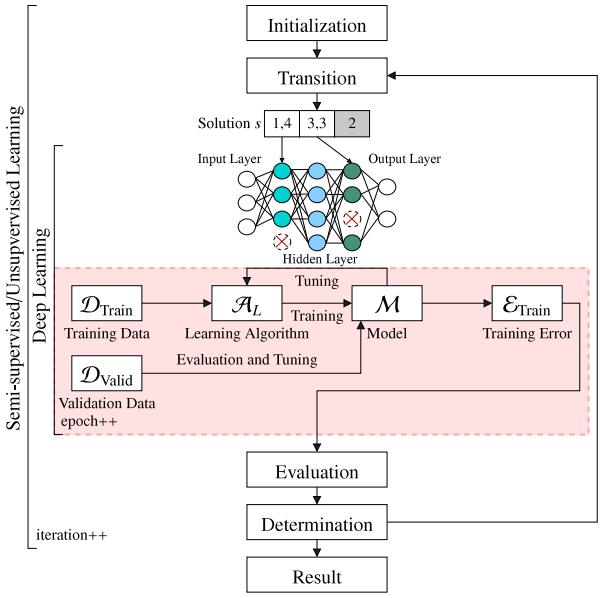


Fig. 1. An example of integrating a semi-supervised or unsupervised search algorithm and deep learning algorithm for neural architecture search.

Such integration of semi-supervised or unsupervised algorithm with deep learning algorithm makes it possible for us to generate a more suitable neural architecture that in turn allows us to build either a more accurate model or a smaller model. Recent literature shows that NAS has been applied to various applications, such as image segmentation [23], object detection [24], image restoration [25], and speaker recognition [26]. Although NAS can be used to automatically generate a suitable neural architecture for the application we are confronted, most of the NAS technologies [20,21] available today require a tremendous amount of computation cost. That is why how to accelerate the process of NAS has become a critical research topic in recent years. Unlike the other recent review papers [8–11,13–15] that first discussed the compression and acceleration technologies for DNN and then attempted to classify these technologies, the focus of this paper

is on both training-free algorithms and their applications to automatic and semi-automatic search algorithms for NAS. We attempted to classify training-free score functions according to whether the method can be used to evaluate different types of model architectures. The benefit of classifying training-free NAS methods in this way is that practitioners can easily choose the optimal score function for different problems. The keywords we used to search for the related literature discussed in this study are "zero-cost" and "training-free". In summary, the contributions of this article can be summarized as follows:

- A brief overview and a framework of the state-of-the-art NAS technologies using training-free score function that are aimed to reduce the complexity of a neural architecture and model are given.
- A brief introduction to the unsupervised learning algorithms for NAS is given. In this respect, the discussion is on using unsupervised learning algorithms with training-free score function instead of a conventional way that requires a training and validation process to reduce or eliminate the computation costs in the estimation of a neural architecture found by a NAS.
- The promising researches and open challenges of training-free algorithms for NAS in which unsupervised or semi-supervised learning algorithms are used are given to infer possible future research directions.

The remainder of the paper is organized as follows. Section 2 begins with a brief discussion of various NAS acceleration methods, the training-free NAS framework, followed by the discussion of model-dependent and model-independent training-free score functions. Section 3 starts with a brief discussion of search algorithms for optimization problems—which include random search and unsupervised learning algorithm—followed by the discussion of search algorithms with training-free score functions for NAS. The focuses of Section 4 are on the measurements of performance evaluation of training-free NAS algorithms and on several well-known benchmarks that have been widely used in evaluating the performance of training-free NAS algorithms in literature to show the way to compare the training-free NAS and non-training-free NAS algorithms. Section 5 gives a set of applications that use training-free NAS algorithms to show their possibility in intelligent systems. Section 6 starts with the open issues, followed by some possible future research directions. Section 7 draws the conclusion of this promising research topic to attract the attention of researchers from different disciplines to develop more efficient NAS algorithms for DNN.

2. Training-free score functions

2.1. From NAS to training-free NAS

The studies in [27–31] show that various neural architectures have been presented in recent years and they all made remarkable achievements in the field of deep learning; however, these neural networks are typically all manually designed by human experts. With the rapid development of these studies, people are trying to apply neural networks to various complex tasks. Since manually designing a neural architecture requires time and effort with preliminary knowledge, it may slow down the progress of designing a machine learning system and increase the cost. Neural architecture search (NAS) methods [32,33] have emerged to deal with such problems by designing neural architectures without human intervention. Also because designing a proper neural architecture with the above traditional NAS methods for a certain task every time requires hundreds or even thousands of GPU hours, several follow-up works started to focus on accelerating the performance of the evaluation process of a candidate neural architecture in NAS. These methods can be categorized into four main areas: (1) lower fidelity estimation, (2) learning curve extrapolation, (3) network transformation, and (4) weight-sharing methods.

In order to expedite the process of evaluating a neural architecture, several works [34–36] attempted to reduce the quality of fully training a model; as such, these methods are usually called "lower fidelity estimation". One common way is to train the neural networks with lower quality images. For instance, the downsampling method can be used to reduce the size of input dataset to cut down the massive training cost caused by large-scale input dataset, e.g., a downsampled dataset ImageNet-16-120 [37] that can be generated from the original dataset ImageNet-1K [38]. Another way is to use only a portion of the input data to train the neural network. Before transferring the architectural block of the neural architecture onto the entire training dataset, Zoph et al. [34] instead search on the partial dataset. But this method has a drawback in case the partial dataset is chosen randomly; that is, it has the risk that the proxy dataset may not be representative enough of the whole original data. For this reason, Na et al. [35] presented a way to construct the proxy dataset by using different selection methods via data entropy. Several studies, such as [36], also attempted to speed up the evaluation of neural networks by training with fewer epochs.

Another way to accelerate the evaluation of a neural architecture is to use a surrogate function or model to predict the performance of the neural architecture. The "learning curve extrapolation" is an efficient method to accelerate the evaluation by predicting the performance of a neural architecture after a few epochs. The training process of a neural architecture will be terminated if the predicted performance is low. Domhan et al. [39] used a probabilistic model to extrapolate the learning curve. Following the work of [39], Klein et al. [40] further improved learning curve prediction by constructing neural architecture with a learning curve layer. Moreover, Baker et al. in their study [41] trained several regression models by a number of features derived from neural architectures, training hyperparameters and early time-series measurements from the learning curve. In contrast to learning curve extrapolation, Liu et al. [42] employed a surrogate model that was trained throughout the search process. The surrogate model was specifically trained based on the properties of small architectures but utilized to predict the performance of larger architectures.

The idea of "network transformation" [43] was first introduced for transfer learning. The basic idea of network transformation is to utilize a meta-controller to explore the search space. The meta-controller will incrementally widen a layer, insert a layer, or add skip connections to the previously searched network. By using this method, the new network will inherit the weights from the previously searched network and train a few more epochs to refine the network. As a result, the weights can be reused by the newly explored network. Such method is also called weight inheritance. For example, Cai et al. [44] employed a reinforcement learning based agent as the meta-controller combined with network transformation for efficient exploration of the search space. Elsken et al. in their study [45] applied a simple hill climbing method as the meta-controller for network transformation with cosine-annealing optimization.

The one-shot technique operates by training a single over-parameterized network during the whole process of exploring the search space. The over-parameterized network, also called supernet, covers the entire search space. When the search algorithm evaluates a network, a subnet is simply sampled from the supernet. Since the weights of the subnet are directly taken from the supernet, such method also refers to "weight sharing". Pham et al. [46] used a recurrent neural network (RNN) as the controller to sample subnets from the supernet. Different from conventional NAS methods, Liu et al. [47] relaxed the discrete search space as a continuous search space that can be optimized by gradient descent. The architecture and weights are jointly learned throughout the search process.

Although the above methods can be used to accelerate the computations of evaluation process of a candidate neural architecture in NAS, most of them, however, still take a lot of computations to evaluate how good a candidate neural architecture is even though the number of computations in this case is much smaller than that required by a complete training process of a candidate neural architecture. That is why some recent studies [48,49] have started with discussions on speedup techniques of NAS, especially the developments from the *n*-shot, few-shot, one-shot, and zero-shot evaluation methods for the evaluation of a neural architecture in NAS. Since these discussions are not specifically on the training-free (zero-shot) NAS and its search algorithm, the following discussion of this paper will focus on these two research topics and relevant research issues.

2.2. The training-free NAS framework

Over the past few years, numerous innovative algorithms were presented to evaluate a neural network architecture without the need of a training process, which significantly reduce the computation time of NAS. Among them, the neural architecture search without training (NASWOT) [50] can be regarded as the pioneer in this field. Fig. 2 provides a general framework of training-free NAS to explain how the search algorithm (e.g., semi-supervised or unsupervised learning algorithm) and training-free score function work together. Same as the framework for traditional NAS algorithms, the *initialization* operator first generates initial solution(s). The *transition* operator then produces new solutions by modifying the current solutions. The *evaluation* operator measures the performance of each candidate neural architecture. The *determination* operator adjusts the search strategy according to the information provided by the *evaluation* operator so that the *transition* operator can generate better new candidate solutions. Different from most existing NAS algorithms that use a complete or partial training process[1] of deep learning to estimate the quality of each candidate solution s (i.e., a neural architecture) generated by the search algorithm, the training-free NAS uses a score function instead to estimate each candidate neural architecture. To make it easier to pinpoint the main difference between existing NAS and training-free NAS algorithms, the counterpart of the part in Fig. 1 that is marked in red is marked in blue in Fig. 2. Note that training data for training-free NAS methods can be either labeled or unlabeled. Also note that the dashed arrow from the training data to the training-free score function in Fig. 2 indicates that the training data might not be used by the training-free NAS method [51]. It is not hard to imagine that a search algorithm will search a large number of candidate neural architectures during the convergence process of NAS. This implies that it will take an enormous amount of computation cost if the training process of DNN has to be applied to each of the candidate neural architectures. For this reason, the training-free score function is typically designed in such a way that it is simple and easy to implement; however, it has to be able to distinguish the "high" accurate from the "low" accurate neural architectures. For example, Wu et al. [52] presented a training-free score function that uses the result of the final layer of a network for an original image o_i and a modified image o_i' to evaluate the performance of a neural architecture. This also implies that the training-free score function takes much less computation cost than using a training process of DNN in the estimation of a neural architecture.

The training-free score function was also referred to as the training-free metric or zero-cost proxy in some recent studies [53–56]. There are several ways to categorize these training-free NASs. One rational way is to divide them into theoretical-based and empirical-based algorithms [57]. As the name suggests, the training-free metrics of the theoretical-based algorithms are constructed based on theoretical findings of neural networks while those of the empirical-based algorithms are constructed based on experimentation or observation. Another way is to classify these training-free NASs based on data usage into two groups, namely, data-dependent and data-independent [58]. The data-dependent algorithm uses the training data to calculate the score of a neural architecture while the data-independent algorithm usually does not require the training data in the sense that it requires only partial information of the training data to score a neural architecture. The last classification again categorizes the training-free NAS algorithms into two groups, that is, model-dependent

[1] By a partial training process of DNN, it means an incomplete training process, such as early stopping algorithm.

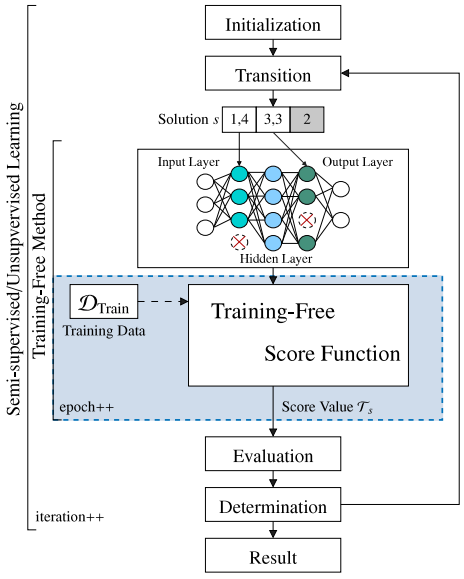


Fig. 2. An example to illustrate the basic idea of training-free NAS.

and model-independent algorithms [57]. The model-dependent score function is limited to certain neural architectures whereas the model-independent score function is applicable to different types of model architectures. The advantage of categorizing training-free NASs in such a way that corresponds to the classification algorithm we use in this paper is that practitioners can straightforwardly decide the type of score function to use for real-world problems.

2.3. Model-dependent score function

These score functions are designed for specific types of neural architectures, such as rectified linear unit (ReLU), CNN, and transformers. Model-dependent score functions are usually optimized for designated neural architectures; therefore, they may have better performance than model-independent score functions. However, when applying model-dependent score functions to evaluating neural architectures that are different from the original design, they may suffer from poor resulting performance, or even worse, the score function does not work at all.

Mean Absolute Error Random Sampling The mean absolute error random sampling (MRS) [59–61] evaluates a recurrent neural network by assessing the expected error performance. The main loop consists of two steps: the weights of the chosen neural architecture are initialized randomly first and then the error is calculated by measuring the difference between real and expected output; i.e., MRS, which is defined as follows:

$$\text{MRS}(\mathcal{D}, \mathbf{h}, t, p_m, Q) = \frac{\phi\left(\frac{p_m-\mu}{\sigma}\right) - \phi\left(-\frac{\mu}{\sigma}\right)}{1-\phi\left(-\frac{\mu}{\sigma}\right)}, \tag{1}$$

where $\mathcal{D}$ is the input dataset; $\mathbf{h}$ the network architecture with Q weights; t the iteration number; p_m the threshold; and Φ the cumulative distribution function (CDF) of the standard normal distribution with mean μ and standard deviation σ. After the termination criterion is met, a probability density function (PDF) is used to fit the recorded errors and rank the architectures by estimating the likelihood of searching for a set of weights that satisfies the defined threshold.

Correlation of Binary Activation Patterns To evaluate how good a ReLU network performs, Mellor et al. [50] first proposed to measure the correlation between binary activation patterns of input data at each ReLU layer. The rationale behind is to see how well the neural network can disentangle two input data. The correlation, which is composed of the binary activation patterns of the whole input data, is defined as the logarithmic determinant of the kernel matrix, as follows:

$$s = \log|\mathbf{K}_H|, \tag{2}$$

where $\mathbf{K}_H$ is the kernel matrix. Besides possessing the ability to distinguish the input data, Lopes et al. [62] further stated that similar input data should have similar scores. Therefore, they considered not only the intra-class correlation but also the inter-class correlation of a network.

Linear Regions Distribution To gauge how complex the function of a ReLU network can represent, Chen et al. [63] and Do and Luong [64] count the number of linear regions a ReLU network is able to be divided. A ReLU network consists of several piecewise linear activations, i.e., ReLUs. The ReLU divides and combines the linear boundaries that are divided by the previous layers. Lastly, the input space of the ReLU network is divided into several linear regions [65]. Because it is infeasible to count the number of linear regions of a large neural network due to its exponential increment, Lin et al. [66], on the other hand, considers the distribution of linear regions by measuring the Gaussian complexity of the linear classifier in each linear region. Due to the gradient explosion problem caused by direct application, they appended batch normalization (BN) layers followed by rescaling to deal with such problem and named such algorithm as Zen-score.

Maximum Entropy Principle In contrast to Zen-score, which requires two forward passes to obtain the score value, Sun et al. [67] proposed to maximize the differential entropy (inspired by maximum entropy principle) by calculating the variance of the output at the final layer with the input Gaussian noise images. However, to deal with the non-uniform object size problem of images, they further used the algorithm called multi-scale entropy that sums up the weighted entropies of the final layer of each stage of a network, which consists of several stages.

Length Distortion Chen et al. [68] designed a score function for vision transformer (ViT) and claimed that a complex neural network is more likely to learn better due to its ability to transform the input into a complex manifold in the final layer. In order to evaluate the complexity of a ViT, a simple input is passed through such a network, and the curvature, length distortion, and length distortion with curvature of the network are measured. They found that the length distortion indicator possesses the highest positive correlation with the lowest computation cost; therefore, the length distortion is used as the training-free score function and is defined as

$$\mathcal{L}^E = \int \sqrt{\|\mathbf{v}(\theta)\|_2} d\theta, \tag{3}$$

where $\mathbf{v}(\theta)$ is the input–output Jacobian of the ViT.

Decoder Parameter Count Albeit many training-free score functions are already faster than the performance estimation strategy of most traditional NASs, most of them still require a forward pass, a backward pass, or both to compute the score. To further cut down the cost of most existing training-free score functions so that they are small enough to be run on the target device, Javaheripi et al. [51] used the number of parameters inside decoders of a transformer network as the budget score function.

2.4. Model-independent score function

Model-independent score functions can be applied to any type of neural architecture. The advantage of this type of score function is that it can be easily adopted for NAS that has a different neural architecture type without the need to redesign the score function.

Gradient-Based Method Abdelfattah et al. [53] proposed several training-free score functions inspired by the pruning-at-initialization literature in which grad norm is a metric that simply calculates the sum of the Euclidean norm of the gradients after propagating a single mini-batch of input data. Besides, Fisher is a metric proposed in [69], which removes the activation channels with minimum impact on loss estimated by performing channel-wise network pruning. Turner et al. [70] further extended the Fisher metric to gauge the importance of a grouped convolution when replacing it with a cheaper alternative. Abdelfattah et al. further integrated and extended the metric to score a whole neural network defined by

$$\mathcal{S}_n = \sum_{i=1}^{M} \mathcal{S}_z(z_i), \tag{4}$$

where $\mathcal{S}_z(z) = \left(\frac{\partial \mathcal{L}}{\partial z} z\right)^2$ is the saliency metric per activation z and M is the length of the feature map. Inspired by [71] when pruning a neural network, SNIP calculates the saliency metric before training with only a single mini-batch of input data. The saliency criterion estimates the loss varied caused by the removal of a particular parameter. The original saliency metric of SNIP, which takes the parameters of a neural network θ as input, is defined as

$$\mathcal{S}_p(\theta) = \left| \frac{\partial \mathcal{L}}{\partial \theta} \odot \theta \right|, \tag{5}$$

where $\mathcal{L}$ is the loss of per-parameter removal and $\odot$ is the Hadamard product. Rather than estimating the change in loss, motivated by [72], GraSP measures the change in gradient norm when pruning the neural network, defined by

$$\mathcal{S}_p(\theta) = -\left(H \frac{\partial \mathcal{L}}{\partial \theta} \right) \odot \theta, \tag{6}$$

where $\mathcal{L}$ is the change in gradient norm and H is the Hessian. SynFlow [73], which intends to prevent layer collapse problem when pruning the neural network, was extended to score the entire network without training. The saliency metric of SynFlow is defined as

$$\mathcal{S}_p(\theta) = \frac{\partial \mathcal{L}}{\partial \theta} \odot \theta, \tag{7}$$

The advantage of this algorithm is that it is data-independent while computing the loss; however, it may suffer from gradient explosion. The score functions extended by Abdelfattah et al. [53]—SNIP, GraSP, and SynFlow—are formulated as a function defined by

$$\mathcal{S}_n = \sum_{i=1}^{N} \mathcal{S}_p(\theta)_i, \tag{8}$$

To deal with the gradient explosion problem of SynFlow, Cavagnero et al. [55] proposed LogSynflow, which simply lessens the gradients by adding a score function prior to aggregating the contribution of each network weight and is defined as follows:

$$\mathcal{S}(\theta) = \theta \cdot \left| \log \frac{\partial \mathcal{L}}{\partial \theta} \right|, \tag{9}$$

Zhang and Jia [57] designed a metric that is founded on the basis of investigating the sample-wise optimization landscape of a neural network. By calculating the signs of the gradients of a mini-batch of input data, they scored a network with such metric called GradSign. The original score function of GradSign is relatively complex; therefore, a simplified formula is used as the score function of GradSign, which is defined by

$$\sum_k \left| \sum_i \operatorname{sign}([\nabla_\theta l(f_\theta(x_i), y_i)|_{\theta_0}]_k) \right|, \tag{10}$$

where x_i is the input feature vector; y_i the input label vector; θ the parameters of the network; θ_0 the parameters randomly initialized; k the vector index; and i the input data index.

Neural Network Gaussian Process The equivalence between infinitely deep neural networks and Gaussian process has been proved in [74] recently. The performance of a neural network can be estimated by neural network Gaussian process (NNGP) [75], which fits the network's kernel regression parameters on a training dataset and assesses its accuracy on a validation dataset. The calculation of NNGP kernel is the main difficulty in computing such score function. Park et al. dealt with this problem by approximating the NNGP kernel by Monte-Carlo algorithm.

Neural Tangent Kernel In a recent study [76], Jacot et al. proposed neural tangent kernel (NTK), which is able to describe the generalization features of a neural network. Chen et al. [63] used the condition number of NTK as the score function to estimate the performance of a network, which is defined as

$$\mathcal{K}_{\mathcal{N}} = \frac{\lambda_{\max}(\hat{\Theta})}{\lambda_{\min}(\hat{\Theta})}, \tag{11}$$

where $\mathcal{K}_{\mathcal{N}}$ is the condition number, $\hat{\Theta}$ is the NTK, $\lambda_{\max}(\hat{\Theta})$ and $\lambda_{\min}(\hat{\Theta})$ are the largest and smallest eigenvalues of NTK between the training input data, respectively. Wang et al. [77]

extended the idea by replacing the NTK with MetaNTK ($\hat{\Phi}$) to adapt to few-shot learning tasks. The new score function can be presented by simply substituting $\hat{\Theta}$ with $\hat{\Phi}$ in Eq. (11). To further extend the works of utilizing NTK as the training-free score function, Shu et al. [78] estimate a neural network by approximating the trace norm of NTK, which provides a better theoretical knowledge of the performance.

Mean of Gram Matrix of Gradients Xu et al. [79] proposed to use Gram matrix of gradients (GM) as a health indicator to see whether a neural network performs well. To be more specific, they leveraged the mean of GM (MGM) to score a network without training, which is defined by

$$\mathbf{g} = \frac{1}{n^2}\sum_{i=1}^{n}\sum_{j=1}^{n}\left(\frac{\partial y_j^{(L)}(t)}{\partial \mathbf{w}(t)}\right)\left(\frac{\partial y_i^{(L)}(t)}{\partial \mathbf{w}(t)}\right)^{\mathsf{T}}, \tag{12}$$

where $\mathbf{g}$ is the MGM score and $\left(\frac{\partial y_j^{(L)}(t)}{\partial \mathbf{w}(t)}\right)\left(\frac{\partial y_i^{(L)}(t)}{\partial \mathbf{w}(t)}\right)^{\mathsf{T}}$ is the dot-product of two gradient vectors of GM.

Noise Immunity Wu et al. [52] proposed a training-free score function, named noise immunity (NI), which evaluates how good a neural network performs by measuring the impact perturbing a mini-batch of input data with Gaussian noise may cause. The NI captures the output vector at the final layer of the network after propagating the perturbed input data and original input data through an untrained network. Then, they estimated the performance of the network by calculating the difference with MSE.

$$n = -\sum_{n=1}^{C}(o_i - o_i')^2, \tag{13}$$

where C is the number of classes; o_i the ith output value of the original input data; and o_i' the ith output value of the perturbed input data. Instead of capturing the outputs at the final layer, Wu et al. in a later study [80] measured the differences of the input data at each downsampling layer. Besides, they also considered two additional factors: the output channel size and the proportion of remaining layers to downsampling layers.

2.5. Hybrid score function

Although a plenty of training-free score functions have been presented in recent years, there are still some critical issues that we have to confront. One is that it is still quite hard to determine which score function is the best. Instead of finding the finest score function, some recent studies [50,52,64,81,82] integrated several metrics to score a neural network. Among them, Wu et al. [52] combined the score function in [50] with the NI they proposed linearly. Instead of directly using the score values of the two metrics, Chen et al. [81] attempted to transform the values into relative rankings and integrated the rankings by simply summing them up. Do and Luong [64], on the other hand, utilized multi-objective optimization algorithm that treats the two metrics as independent objectives. Tran et al. [82] combined four training-free indicators by using a fusion framework in which the fusion indicator (FI) is defined as

$$S_{\text{FI}} = \sum_{i=1}^{n} w_i S_i + b, \tag{14}$$

where S_i is the score of the ith indicator, n the number of indicators, w_i the weight for the ith indicator, and b the bias.

Another critical issue is that although the results of most recent studies on the accuracy of training-free NAS algorithms are quite close to or even better than those of non-training-free NAS algorithms, most existing training-free score functions still cannot completely surpass the training process of DNN for all datasets because most of them are based on estimation or inference instead of actually training the network architecture. It is just like estimating the weight of a product by the naked eye, which is faster but not as precise as using a scale. Fortunately, a rough estimation of each solution is generally good enough during the convergence process of some search strategies (e.g., metaheuristic algorithm) because the basic idea is to select relative better solutions in the end of the convergence process. This implies that the search algorithm has a strong impact on the final results of NAS; hence, an effective search algorithm with suitable estimation method should be capable of finding a good neural architecture. From this observation, it can be easily seen that another critical issue in the development of an effective training-free NAS is obviously how to design an appropriate search algorithm.

3. Search algorithms

Figs. 1 and 2 show that the two essential parts of most non-training-free and training-free NASs are no doubt the search algorithm and estimation method. An intuitive way to choose the search algorithm for finding a suitable neural architecture in NAS can be a random or greedy search algorithm. If NAS is considered as an optimization problem, we can then assume that the goal of the optimization problem is to find the best neural architecture s^* from a set of candidate neural architectures S defined by

$$s^* = \max_{s\in S}\mathcal{L}(\mathcal{A}_s(\mathcal{E}(s, \mathcal{D}_{\text{Train}})), \mathcal{D}_{\text{Test}}), \tag{15}$$

where $\mathcal{L}()$ is a measurement function; $\mathcal{A}_s$ the accuracy of the searched result from the search algorithm; $\mathcal{E}()$ the estimation method for a neural architecture which can be either a non-training-free or training-free method; $\mathcal{D}_{\text{Train}}$ the training data, and $\mathcal{D}_{\text{Test}}$ the testing data. Since NAS can be regarded as an optimization problem, in addition to the score function (i.e., estimation method), the search ability of the selected search algorithm will also have a strong impact on the end results of the training-free NAS algorithm. For this perspective, it can be easily seen why semi-supervised (e.g., reinforcement learning) and unsupervised learning (e.g., metaheuristic algorithm) algorithms, such as random search (RS), Bayesian optimization (BO), differentiable architecture search (DARTS), and metaheuristic algorithm, have been used with the training-free NAS in recent studies.

3.1. From exhaustive search to metaheuristic algorithms

Before we move on to search algorithms for NAS, a brief introduction to the search algorithm for solving optimization problem will be first given in this section. From the development to solve optimization problems (e.g., large-scale

optimization problems or NP-hard problems), several traditional exhaustive search algorithms can be used to exactly solve an optimization problem (e.g., dynamic programming (DP)). Of course, most researchers in computer science or relevant research domains know that although exhaustive search algorithms are capable of finding the optimum solution of an optimization problem; however, they are not able to exactly solve the optimization problem within a reasonable time, especially for complex problems.

To find an "appropriate solution" within a reasonable time, a set of greedy search algorithms have been presented for solving optimization problems, such as hill climbing (HC). An appropriate solution here is also called an approximate solution in most greedy search algorithms while some of the greedy search algorithms are also called heuristic algorithm. By an approximate solution, it means that the solution is close to but might not be the optimal solution. As a well-known greedy search algorithm, the basic idea of search strategy of HC is to accept only a better solution as the new or current solution. This is why the greedy search and heuristic algorithms may get stuck at a local optimum during the convergence process if no better solutions can be found around this local optimum [83]. The search strategy of HC can also be used to explain that the greedy and heuristic algorithms adopt one or more rules as the determination operator to determine the search directions or regions in later iterations during the convergence process.

The metaheuristic algorithms provide an alternative way to prevent the searches from easily falling into local optima during the early convergence process. It is generally believed that the term "metaheuristic" is coined by Glover [84], which is formed from two Greek words—"meta" and "heuristic". Like the heuristic algorithm, a metaheuristic algorithm still attempts to find the approximate solution(s) within a reasonable time; unlike the heuristic algorithm, most metaheuristic algorithms are typically embedded with one or more mechanisms to make it possible for the searches to be able to escape from a local optimum. The main design of a metaheuristic algorithm is just like other semi-supervised and unsupervised algorithms contains five crucial operators that are initialization, transition, evaluation, determination, and output, as mentioned in Section 1. Because of the mechanism(s) of a metaheuristic algorithm to avoid the searches from falling into local optimum, most metaheuristic algorithms are able to find better results than greedy search and heuristic algorithms in most cases [83,85]. That is why a large number of studies attempted to use metaheuristic algorithms to solve optimization problems from the 1990s or even earlier. These algorithms [86] include tabu search (TS), simulated annealing (SA), genetic algorithm (GA), differential evolution (DE), ant colony optimization (ACO), and particle swarm optimization (PSO). Their development makes metaheuristic algorithms become one important branch in the studies of unsupervised learning algorithm.

Since metaheuristic algorithms provide an efficient way to find approximate solution(s) in solving an optimization problem, it is not difficult to imagine that several metaheuristic algorithms have then been applied to enhance the performance of NN [87–89] and DNN in studies [16,17,90]. From the studies of [88,89], it can be found that GA can be used to find suitable features for NN [88] or learn connection weights [89] for NN. Some recent studies [16,17,90] have shown that metaheuristic algorithms can also be used to improve the end results of a DNN, such as selecting a set of good hyperparameters for DNN or finding a suitable neural architecture of DNN. All these successful results show that other metaheuristic algorithms may be able to enhance the performance of an NN and DNN. In the later discussions, we will use metaheuristic algorithms for hyperparameter optimization (HPO) and neural architecture search (NAS) as two examples to explain why a search algorithm is so important for the DNN. As shown in Fig. 3(a), a search algorithm (e.g., a semi-supervised or an unsupervised learning algorithm) can be used to find a good plan for the DNN by embedding the DNN as the measurement to evaluate how good a set of hyperparameters, a neural architecture, or a design of DNN is. Fig. 3(b) and (c) show that a search algorithm can be used to find a set of hyperparameters or a neural architecture ($\mathcal{N}$) if they can be encoded into a feasible solution of the search algorithm.

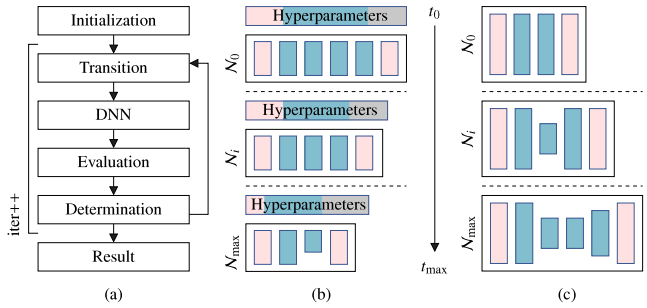


Fig. 3. Two examples of a search algorithm for hyperparameter optimization and neural architecture search. (a) The design of a search algorithm to enhance the performance of DNN. (b) How a search algorithm finds a set of good hyperparameter. (c) How a search algorithm finds a good neural architecture. Note that $\mathcal{N}_i$ represents the best-so-far neural architecture found by a search algorithm at iteration t_i.

Because the performance of a DNN will be strongly impacted by the hyperparameters (e.g., learning rate, number of hidden layers, number of neurons in each hidden layer, and so forth), the goal of HPO is to find a set of "good" hyperparameters for DNN to further improve its end results. The grid search algorithm [91] is a well-known method for searching a set of suitable parameters for a search algorithm which has been widely used in several studies for years. Since the basic idea of grid search is simple and easy to implement, it can thus be used to search a set of suitable hyperparameters for DNN and can be regarded as an example of the rule-based algorithm. However, in the studies of [92,93], Bergstra et al. showed that random search algorithm is able to find better hyperparameters for DNN than grid search. The later studies of [94,95] further showed that heuristic-based algorithms (e.g., Bayesian optimization algorithm) can also find better hyperparameters for DNN than grid search. In addition to grid search, random search, and heuristic search algorithms, some recent studies attempted to use PSO-based algorithms [96,97]

to solve HPO, which shows that metaheuristic algorithms provide an effective way to improve the performance of DNN.

Another representative example to use the metaheuristic algorithm for DNN is the neural architecture search (NAS) [98]. In this research topic, the metaheuristic algorithm typically can be used to design a compact neural network architecture to accelerate the training and inference speed of a DNN or to find a good neural architecture to improve the end results of a DNN. We can also use a reinforcement learning algorithm as the search algorithm for NAS. Cai et al. [44] proposed an efficient framework for DNN design using reinforcement learning. By leveraging a reinforcement learning agent as a meta-controller to find a more accurate and compact neural architecture, a large amount of computational cost can be reduced. Apart from reinforcement learning, Lin et al. [99] used the artificial bee colony algorithm (ABC) to remove a certain number of feature map channels to reduce the size and computation costs of the whole neural architecture. So the computation cost of a DNN can then be reduced. In the study of [100], Zhou et al. attempted to define this problem as a multi-objective optimization problem so that the search algorithm is able to find a suitable CNN neural architecture that has not only better accuracy but also smaller size of neural architecture compared with CNN. In this study, a knee-guided evolutionary algorithm is used as the search algorithm for solving such a multi-objective problem. It is apparent that a good search algorithm can also be used to find a good neural architecture to improve the accuracy of a DNN. As an important branch of metaheuristic algorithms, the evolutionary algorithm (EA) has been widely used for the NAS in recent studies [101–104]. The role of an EA for NAS is to generate a set of new candidate neural architectures, to evaluate these new candidate neural architectures v, and then to determine and select a certain number of good new candidate neural architectures v' for later iteration to continuously search for new candidate neural architectures around the neural architectures v'. The results of [101–104] showed that metaheuristic algorithms are capable of finding better neural architecture except that they take much more additional computation costs.

3.2. Search algorithms for training-free NAS

3.2.1. Random search

Although the random search (RS) does not fall into the groups of semi-supervised and unsupervised learning algorithm, it is simple, intuitive, and easy to implement for solving an optimization problem. That is why it has been the first choice when solving an optimization problem, and its results can be used as the baseline for the purpose of comparison. In general, RS can often obtain descent results for small NAS search spaces [50,62]. However, when applying RS as the search strategy to a relatively larger search space (e.g., NAS-Bench-101), the accuracy is typically not as good as that of using RS for small search spaces (e.g., NAS-Bench-201) [50].

3.2.2. Bayesian optimization

Bayesian optimization (BO) is an optimization technique that leverages Bayes Theorem to sequentially guide the search without the need of calculating the derivative of the objective function. Camero et al. [60] utilized BO as the search strategy of their training-free NAS and tackled the variable-length solution problem by proposing three different encoding schemes that transform the variable dimensions of NAS into fixed dimensions.

3.2.3. Reinforcement learning

Reinforcement Learning (RL) is composed of a dynamic environment and an agent. The agent interacts with the environment by taking actions, and the environment gives a reward in response to the agent according to the action made by the agent. By trial-and-error to adjust the search direction of the agent, it can eventually find the optimal solution. Tran and Bae [105] leveraged RL as the search strategy to guide the search of their training-free NAS. They also defined a reward function that maximizes the score and maintains the latency under an expected latency at the same time, as follows:

$$R(m) = S \times R_{\mathrm{LAT}}(m), \tag{16}$$

where m is a network, S is the training-free score function, and R_{LAT} is the reward for latency defined by

$$R_{\mathrm{LAT}}(m) = \begin{cases} \frac{T}{\mathrm{LAT}(m)}, & \text{if } \mathrm{LAT}(m) > T, \\ g(\mathrm{LAT}(m)), & \text{otherwise}, \end{cases} \tag{17}$$

where T is the expected latency, $\mathrm{LAT}(m)$ is the latency of network m, and g is a linear reward function.

3.2.4. Differentiable search algorithm

Differentiable search algorithm (DSA) is an optimization algorithm designed for NAS by relaxing the constraint from a discrete search space to a continuous search space; therefore, the problem can be optimized directly by gradient descent. Recent training-free NAS studies have utilized a differentiable search algorithm called differentiable architecture search (DARTS) as the search strategy [54,106]. Because only one neural network (i.e. supernet) is trained during the search of DARTS, it is trivial to directly replace the training process with training-free score function. Zhang et al. [106] instead proposed a framework for training-free NASs based on DARTS that scores the operations in the supernet with the training-free score functions, called operation saliency metrics.

3.2.5. Metaheuristic algorithm

Heuristic algorithms, such as hill climbing, have been used in NAS applications. For instance, Elsken et al. [45] adopted hill climbing procedure to search for CNNs with cosine annealing optimization. Metaheuristic algorithms are higher level heuristics that can be used to optimize complex optimization problems. A subgroup of the metaheuristic algorithms are inspired by biological mechanisms and are called evolutionary algorithms (EAs). A plenty of training-free NAS studies have leveraged EAs as their search strategy [51,52,61,67,82]. Wu et al. [52] utilized genetic algorithm (GA), which is one

of the most popular evolutionary algorithms, as their search strategy. The tournament selection is used to determine the search direction of the architectures. Tran et al. [82] and Cavagnero et al. [55] used a recently proposed evolutionary algorithm, aging evolution [36], to replace the tournament selection by preserving the best architecture and removing the oldest one. Do et al. [64] utilized NSGA-II, which is a popular multi-objective evolutionary algorithm, as the search strategy to optimize two training-free score functions simultaneously. Arman and Deowan [107] proposed an improved version of an existing metaheuristic algorithm that is inspired by the leadership behavior of grey wolves, grey wolf optimization (GWO), by replacing the ranking metric by a saliency metric. Mokhtari et al. [56] proposed an improved version of another metaheuristic algorithm that mimics the flashing pattern and behavior of fireflies, i.e., firefly algorithm (FA). The improved FA is combined with the operators of GA to give it a better ability to avoid local optima. In addition, the fitness function is replaced by a training-free score function. From the end results of these studies, it can be easily seen that the training-free score function is a crucial technology for metaheuristic-based NAS. One important reason is that metaheuristic algorithms are designed to spend much more computation costs than rule-based algorithms (e.g., random search and greedy search algorithms) although metaheuristic algorithms themselves may be able to find better results than rule-based algorithms for solving optimization problems, and the computation cost to evaluate a neural architecture in a metaheuristic algorithm will dramatically increase for the NAS. Thus, the training-free score function can be used for a metaheuristic algorithm to reduce most computation costs for the NAS to make it possible for a metaheuristic algorithm to find a better neural architecture faster.

3.2.6. Hybrid heuristic algorithm

Although EAs provide an efficient way to search for optimal solutions, most of the evolutionary algorithms for training-free NAS searching only in a discrete search space results in a relatively immense amount of computational cost. DARTS, on the other hand, turns the discrete search space into a continuous search space; however, the search space is usually tremendously large due to the supernet. One way to accelerate the search process of training-free NAS is to combine different search strategies. Geada and McGough [108] proposed a hybrid search strategy by integrating EA and DARTS. Instead of configuring a supernet and performing pruning, they start with a small supernet and consecutively expand the search space by EA, which greatly reduces the search cost of training-free NAS.

4. Performance evaluation

4.1. Metrics

Efficiency: Even though training-free methods do not suffer from the issue of heavy computation costs to train a neural network, several score functions require complex mathematical operations, such as matrix multiplication. To enable NAS algorithms to search on edge devices with limited computing resources, cutting down the search time to the utmost is still the current top priority task. Since these methods may be evaluated under different hardware setup (e.g. multiple GPUs), it is important to impartially assess the efficiency of the training-free NAS algorithms. Suppose the search time is only reported in seconds, minutes, or hours, the comparisons between these methods may be unfair. As a result, it is more sensible to provide the search cost in GPU time.

Effectiveness: To evaluate the effectiveness of a neural architecture, one approach is to calculate the correlation between the predicted score and ground-truth test accuracy. There are several metrics that can be used to measure the correlation. Pearson correlation coefficient is a simple way to measure the linear relationship between the predicted score (x) and ground-truth test accuracy (y). It is a value between -1 and 1, where 1 indicates a perfect positive correlation, -1 indicates a perfect negative correlation, and 0 indicates no relationship, defined as

$$\rho_{\text{pearson}} = \frac{\sum_{i=1}^{n}(x_i - \mu_x)(y_i - \mu_y)}{\sqrt{\sum_{i=1}^{n}(x_i - \mu_x)^2 \sum_{i=1}^{n}(y_i - \mu_y)^2}}, \tag{18}$$

where μ_x is the mean of x and μ_y is the mean of y. Kendall rank correlation coefficient and Spearman rank correlation coefficient on the other hand measure the monotonic relation. Same as Pearson coefficient, they will return a value between -1 and 1 and are defined as

$$\rho_{\text{kendall}} = \frac{(P_c) - (P_d)}{0.5n(n-1)}, \tag{19}$$

and

$$\rho_{\text{spearman}} = 1 - \frac{6\sum(x_i - y_i)^2}{n(n^2-1)}, \tag{20}$$

where P_c is the number of concordant pairs and P_d is the number of discordant pairs.

Despite a higher correlation between the predicted score and ground-truth test accuracy implies a better training-free score function, the end results may not always be consistent when the score function is combined with an effective search algorithm. Here, Kendall rank correlation coefficient is used as the score function of NASWOT [50] with a search algorithm as an example to analyze the performance of the training-free NAS.

Let us further assume that an effective search algorithm $\mathcal{A}_s$ will trend to search region A, and a relatively ineffective search algorithm $\mathcal{B}_s$ will trend to search region B. As shown in Fig. 4, albeit it shows a higher correlation between the predicted score and ground-truth test accuracy, the score function $\mathcal{F}$ performs better only when a relatively effective search algorithm is utilized to guide the search process within the region A. More precisely, the score values of some disrepute neural architectures in region B will be misjudged as quite high as other good neural architectures in some cases. For this reason, the role of a metric (e.g., Kendall rank correlation coefficient) is to provide some information to analyze the results, but we cannot use only one metric to make a biased conclusion. In this example,

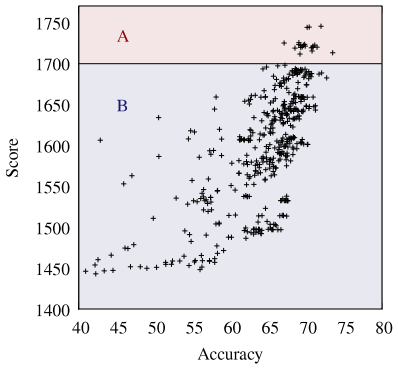


Fig. 4. The relationship between the predicted score and ground-truth test accuracy of 500 randomly selected architectures.

it can be easily seen that a score function $\mathcal{F}$ will be performed better when an effective search algorithm is utilized. The top-N measurement is another way to evaluate the effectiveness of a training-free score function. N architectures that have the highest score are selected and trained. The top-N accuracy is the highest test accuracy among the N architectures. Mostly, top-1 accuracy is a more reasonable and practical way to assess the effectiveness of a neural architecture since training a network is usually very costly. The stability is also a crucial measurement when evaluating the effectiveness of a training-free score function; thus, accuracy and standard deviation are typically considered together to measure the stability of a method.

4.2. Benchmarks

Since many training-free NASs have sprung up in recent years, how to evaluate their performance has become an important research topic. To enable a fair comparison between these algorithms, a benchmark that provides pre-trained network weights and information of the performance of each network architecture in the search space is necessary. Predefining a common search space is also essential to provide an unbiased comparison. The most intuitive way is to construct a global search space that has a relatively few restrictions to the combination of neural operations. If the neural architectures of the search space consist of only one path from input to output, it is called a chain-structured search space as shown in Fig. 5(a). Otherwise, it is called a multipath-structured search space. As shown in Fig. 5(b), the neural architecture can usually be represented as a directed acyclic graph (DAG). Some nodes have multiple input from different previous nodes. In such a case, there are mainly two different ways to combine these input. One is to concatenate the input while the other is to sum the input element-wise. Since global search spaces confine the combination of operations in a lower degree, NAS algorithms will often have to spend massive computing cost. A simple way to deal with such problem is to define a common macro structure shared among all the neural architectures of the search space and only allow the operations inside the cells to be changed. This kind of search space is called cell-based search space, as shown in Fig. 5(c).

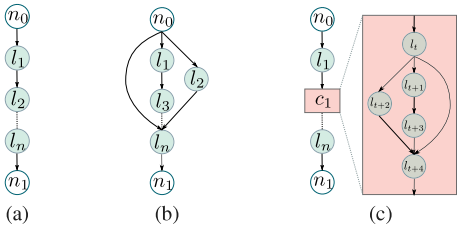


Fig. 5. Three common search space in NAS literature. (a) The chain-structured search space. (b) The multipath-structured search space. (c) The cell-based search space.

By using a common benchmark, researchers neither have to worry about differences in the training results that may be caused by different training settings nor have to train the same network repetitively; therefore, they can focus on optimizing the performance of the algorithm. Because NAS benchmarks have sprung up that focus on different types of architectures and tasks, a summary of these benchmarks is given in Table 1.

Several popular benchmarks [47,109,110] have already been utilized in the literature of many training-free NASs. NAS-Bench-101 [109] is one of the earliest NAS benchmarks, which consists of 423 K unique architectures. In contrast to NAS-Bench-101 that is only trained on one dataset, NAS-Bench-201 [110] is trained on multiple datasets and provides more elaborate information of networks; however, the search space, with merely 6466 unique topological structures, is relatively small. In a later study, [111] Dong et al. augmented the search space with a size search space that varies the number of channels with a single architecture topology, called NATS-Bench. The DARTS search space [47] is well-known for the continuous feature that enables being searched by gradient descent. The search space consists of 10^{18} architectures and each searched network architecture has to be trained from scratch. NAS-Bench-301 [112] provides a surrogate benchmark that predicts the architecture ground-truth performance of all the architecture of DARTS search space. NAS-Bench-1Shot1 [113] is a benchmark provided for evaluating one-shot NAS algorithms. While the above NAS Benchmarks are designed for image classification tasks, NAS-Bench-NLP [116] is a benchmark for natural language processing (NLP) that consists of 14k architectures. Compared to the aforementioned benchmarks that are for single-task, TransNAS-Bench-101 [118] is a special benchmark that covers seven different tasks, including classification, regression, pixel-level prediction, and self-supervised tasks. NAS-Bench-360 [121] is also a benchmark that focuses on many different tasks. Some of the datasets are new to the NAS research field. Among them are Spherical CIFAR-100 [124], NinaPro [125], Cosmic [126], and Satellite [127].

The benchmarks mentioned above typically consist of unprecedented structures in state-of-the-art manually-designed deep learning algorithms; therefore if a NAS algorithm is designed to search on these benchmarks, its neural architecture and model may not work well when applying to real-world applications [122]. This leads to a gap between these benchmarks and real-world designs. Another concern in utilizing

Table 1
The summary of various NAS benchmarks.

Benchmark	Task	Dataset	Search space	Architecture	Ref.
NAS-Bench-101	Image classification	CIFAR-10	Cell	423.6k	[109]
NAS-Bench-201	Image classification	CIFAR-10, CIFAR-100, and ImageNet-16-120	Cell	6.5k	[110]
NATS-Bench	Image classification	CIFAR-10, CIFAR-100, and ImageNet-16-120	Cell	6.5k + 32.8k	[111]
NAS-Bench-301	Image classification	CIFAR-10	Cell	10^{18} + 60k	[112]
NAS-Bench-1Shot1	Image classification	CIFAR-10	Cell	14.6k	[113]
NAS-Bench-NLP	Language modeling	PTB [114] and WikiText-2 [115]	Cell	14k	[116]
TransNAS-Bench-101	7 tasks (object classification, scene classification, semantic segmentation, etc.)	Taskonomy [117]	Cell	7.3k	[118]
NAS-Bench-360	10 tasks (image classification, disease detection, DNA sequences prediction, etc.)	CIFAR-100, ECG [119], DeepSEA [120], etc.	Cell	15.6k	[121]
NAS-Bench-Zero	Image classification	CIFAR-10, CIFAR-100, and ImageNet-16-120	Chain	14.9k + 10.1k + 9.7k	[122]

Note that CIFAR-10 and -100, and ImageNet-16-120 are from [123], and [37].

these benchmarks for training-free NAS techniques, pointed out by Ning et al. [128], is that the final test accuracy of a neural network on these benchmarks has a strong correlation with the number of parameters, which may cause people to design their training-free score function according to these special patterns that do not exist in practice. To deal with the above problems, Chen et al. [122] designed a benchmark for training-free NAS algorithms called NAS-Bench-Zero. NAS-Bench-Suite-Zero [129] is another benchmark for training-free NAS, which covers 28 different tasks and is the largest dataset as of today.

Table 2 shows the comparisons between some non-training-free and training-free NASs based on the results of our previous work [80]. From these comparisons, it can be easily seen that existing non-training-free NAS can get from 83.74% up to 93.34% top-1 accuracy. Both REA [33] and ENAS [130] outperformed DARTS [47] in terms of accuracy. REA [33] takes 12,000 s to obtain about 93% accuracy. Although ENAS [130] can achieve the highest top-1 accuracy 93.34% among all the methods, it is the most inefficient algorithm that takes 22,325 s. The results of NASWOT [50] and TNASSE [80] show that a training-free NAS can significantly reduce the search time of a NAS down to only hundreds of seconds. The test results demonstrate that training-free NAS methods can achieve competitive top-1 accuracy results, with TNASSE reaching an impressive accuracy of 92.50%. Although these training-free methods may not surpass the performance of REA, they come remarkably close. Moreover, the significant difference in search time between TNASSE and REA indicates a substantial acceleration of 34.58 times or a reduction of 97.11%. These findings highlight the potential of training-free NAS algorithms to achieve comparable accuracy while significantly reducing the computational burden associated with traditional methods like REA.

5. Applications

As artificial intelligence applications are becoming more and more complex, designing an optimal neural architecture for a certain task using minimum computing resources has become a critical issue to be resolved. Because training-free NAS methods do not require training neural networks during the search process, they have been used for various applications, such as computer vision and natural language processing tasks. In this section, we use some applications as examples to show the possibility of training-free NAS methods.

5.1. Computer vision

Computer vision extracts information from image data or video data to distinguish data, take actions, or even make decisions. Since the neural network training for computer vision tasks is becoming more and more time-consuming due to large-scale image data, applying NAS to these tasks has become more and more challenging; thus, the training-free NAS algorithm has become a promising solution. The most common application seen in various training-free NAS papers [50,52,53,55,57,62,63,66–68,75,77–79] is image classification. The goal of image classification is to accurately classify a given input image to the category it belongs. Besides, object detection is also a popular field in computer vision, which builds on image classification that identifies the target object in images or videos. Using NAS methods to search for optimal neural architectures for object detection tasks is extremely computationally costly. Although training-free NAS method is a possible way to deal with the tremendous computation required, it has only been explored in a few papers [67,68].

5.2. Natural language processing

Natural language processing (NLP) is another field in artificial intelligence that extracts and analyzes information from

Table 2
Comparisons between non-training-free and training-free NAS algorithms [80].

Algorithm	Evaluation method	Search algorithm	Top-1 Acc.	Cost (s)	Ref.
Non-training-free NAS					
REA	Low-fidelity	Evolutionary algorithm	93.02	12,000	[36]
DARTS	One-shot	Differentiable search algorithm	83.74	20,483	[131]
ENAS	One-shot	Reinforcement learning	93.34	22,325	[130]
Training-free NAS					
NASWOT	Correlation of binary activation patterns	Random search	91.95	140	[50]
TNASSE	Noise immunity	Evolutionary algorithm	92.50	347	[80]

Note that the top-1 accuracy and cost are reported for NAS-Bench-101. The results are conducted on a single NVIDIA RTX2080 Ti GPU.

text data. One popular area in NLP is language modeling, which learns rules from the features of natural language and applies it to make prediction or generate new text. In [53], a training-free function (called zero-cost proxy) is used to search for a new recurrent model to speed up the NAS for NLP. Recently, many language models, such as BERT [132] and GPT3 [133], have utilized transformer architecture for building the model. Since each transformer layer has different architectural settings, NAS for transformer architectures may cause unaffordable computation costs. That is why Javaheripi [51] made use of training-free NAS algorithms for transformer architecture search.

5.3. Tiny device

With the advance of technology and the requirement of deploying neural networks on tiny devices, shrinking down the model size while maintaining the prediction accuracy simultaneously is a trending topic in machine learning. Current neural networks deployed on tiny devices are mostly designed by human. Since machine learning systems are becoming more and more complex, edge devices require neural networks to perform certain tasks. Training-free NAS methods can greatly reduce the computing time required to design a suitable neural architecture to be placed on edge devices compared to traditional NAS. Sun et al. [67] have attempted to find a neural architecture using training-free NAS and test on a Google Pixel 2 phone. According to our observations, applying training-free NAS algorithms to search for compact neural architectures is still in its rudimentary stage.

6. Open issues and future directions

Since training-free NAS is still in its early development stages, there are several open research issues that need to be addressed to develop high-performance training-free NAS algorithms for DNNs. These open research issues can be summarized as follows:

- **Accuracy Limitation**: Although the end results of most training-free NAS algorithms are typically close to those of non-training-free NAS algorithms, there is plenty of room for improvement in the development of more accurate score function for one reason. Training-free score functions might misjudge the searched neural architecture found by a search algorithm.
- **Generalization to Different DNNs**: Some training-free NAS algorithms are designed for a particular type of DNN or model. How to apply a good training-free NAS algorithm to different DNN or model if it can only perform well for the particular DNN is also an important open issue today.
- **Transferability to Different Benchmarks**: Another open issue is caused by some training-free NAS algorithms that are designed for particular benchmarks. The reason is that candidate neural architectures in a benchmark will follow particular rules and structures; thus, the training-free score function that is designed for these rules will become a dilemma when we want to apply a training-free score function to different benchmark.
- **Real-World Performance**: Since most training-free NAS algorithms are used for well-known datasets to show their performance, but not for real-world datasets. Whether or not they still have the similar performance compared with non-training-free NAS algorithms for real-world datasets has become an open issue now.
- **Theoretical Foundations**: Until now, although many statistical analyses have been done for training-free NAS algorithms, there still lack theoretical analysis of training-free score function. A complete theoretical analysis of such score function is needed to support this research field.

From these open issues, it can be easily seen that the number of datasets is still limited today. It is easy to understand that training-free NAS provides an alternative way to reduce the computation costs of NAS; its possibilities, however, are hard to imagine. The main reason is that the research topic of training-free NAS is new and still in the early stages of development. That is why only a few well-known datasets were used in most studies. The same situation can also be found in the applications of training-free NAS, which are also currently constrained. It can be expected that more datasets will be presented in the coming future, or we can apply it to different applications to be pioneers in this research topic. In addition to the open issues of training-free NAS, as the researchers in this research field, how to enhance the performance of training-free NAS based algorithms and how to apply them to the practical applications in our daily life are also crucial research directions. To help the audience of this paper find some *interesting research topics*, the possible future research trends are given below:

Table 3
Comparisons between training-free NAS algorithms.

Algorithm	Performance		Training-free score function	Search algorithm	Benchmark	Dataset	Ref.
	Acc.	ρ					
Model-dependent							
MRS	–	–	MAE random sampling	Random search	–	Sine Wave, Predicting Parking Occupancy Rates [134], and Appliances Energy Consumption [135]	[59]
BOMRS	–	–	MAE random sampling	Bayesian optimization	–	Sine Wave, Waste [136], and Load Forecast [137]	[60]
RESN	–	–	MAE random sampling	Evolutionary algorithm	–	Sine Wave, Waste [136], Coal-fired Problem, and Load Forecast [137]	[61]
NASWOT	92.96	$[0.57]_k$	Correlation of binary activation patterns	Random search	NAS-Bench-101, NAS-Bench-201, NATS-Bench [111], and NDS [138]	CIFAR-10, CIFAR-100, and ImageNet-16-120	[50]
EPE-NAS	91.31	–	Correlation of binary activation patterns	Random search	NAS-Bench-201 [110]	CIFAR-10, CIFAR-100, and ImageNet-16-120	[62]
TE-NAS	93.9	$[0.50]_k$	Linear regions distribution	Differentiable search algorithm	NAS-Bench-201 and DARTS [47]	CIFAR-10, CIFAR-100, and ImageNet-16-120	[63]
Zen-NAS	–	–	Linear regions distribution	Evolutionary algorithm	–	CIFAR-10, CIFAR-100, and ImageNet-16-120	[66]
MAE-DET	–	–	Maximum entropy principle	Evolutionary algorithm	–	COCO [139] and ImageNet-1k [38]	[67]
As-ViT	–	–	Length distortion	Reinforcement learning	–	COCO [139] and ImageNet-1k [38]	[68]
LTS	–	–	Decoder parameter count	Evolutionary algorithm	–	WikiText-103 [115] and One Billion Word [140]	[51]
Model-independent							
Zero-Cost NAS	–	$[0.82]_s$	Gradient-based	Random search, Reinforcement learning, and Evolutionary algorithm	NAS-Bench-101, NAS-Bench-201, NAS-Bench-NLP [116], and NAS-Bench-ASR [141]	CIFAR-10, CIFAR-100, ImageNet-16-120, WikiText-103 [115], PTB [114], and TIMIT [142]	[53]
FreeREA	94.35	$[0.63]_k$	Gradient-based	Evolutionary algorithm	NAS-Bench-101 and NATS-Bench [111]	CIFAR-10, CIFAR-100, and ImageNet-16-120	[55]
GradSign	93.31	$[0.77]_s$	Gradient-based	Random search	NAS-Bench-101, NAS-Bench-201, and NDS [138]	CIFAR-10, CIFAR-100, and ImageNet-16-120	[57]
NNGP-guided NAS	–	–	NNGP	Random search	NAS-Bench-101 and MNAS [143]	CIFAR-10 and ImageNet-1k [38]	[75]
TE-NAS	93.9	$[-0.42]_k$	NTK	Differentiable search algorithm	NAS-Bench-201 and DARTS [47]	CIFAR-10, CIFAR-100, and ImageNet-16-120	[63]
MetaNTK-NAS	–	–	NTK	Differentiable search algorithm	–	mini-ImageNet [144] and tiered-ImageNet [145]	[77]
NASI	–	–	NTK	Differentiable search algorithm	NAS-Bench-1Shot1 [113] and DARTS [47]	CIFAR-10, CIFAR-100, and ImageNet-1k [38]	[78]
KNAS	93.43	$[0.56]_s$	MGM	Random search	NAS-Bench-201	CIFAR-10, CIFAR-100, and ImageNet-16-120	[79]

(*continued on next page*)

Table 3 (*continued*).

GA-NINASWOT	93.70	–	Noise immunity	Evolutionary algorithm	NAS-Bench-201	CIFAR-10, CIFAR-100, and ImageNet-16-120	[52]
TNASSE	93.41	$[0.49]_k$	Noise immunity	Evolutionary algorithm	NAS-Bench-101, NAS-Bench-201, and NATS-Bench [111]	CIFAR-10, CIFAR-100, and ImageNet-16-120	[80]

Note that NAS-Bench-101, NAS-Bench-201, CIFAR-10 and -100, and ImageNet-16-120 are from [37,109,110,123], respectively. Also note that the Acc. indicates the top-1 accuracy for CIFAR-10 dataset, and the ρ indicates the correlation of training-free score function and accuracy. Moreover, the subscripts s, p, and k in the ρ field represent the values of Spearman, Pearson, and Kendall correlation, respectively.

- **Computation Cost and Storage Size**: Although NAS can be used to reduce the model size of DNN and training-free NAS can also be used to accelerate the running time, if a DNN model exceeds the peak limit of memory or storage, its performance on tiny devices [146,147] will be affected. For this reason, this research direction can be the next step of training-free NAS development.
- **Performance of Search Algorithm**: The search algorithm plays an important role in the training-free NAS because its performance will affect the computation time and end result of NAS. How to enhance the performance of the search algorithm to make the best use of the advantages of training-free NAS will be very useful for enhancing the performance of a NAS.
- **Decentralized Computations**: Most of the existing training-free NAS algorithms are designed on a centralized computing environment, just like most NAS algorithms. Decentralized searches of all neural architectures provide an alternative way to enhance the performance of the training-free NAS algorithm, especially for large-scale datasets.
- **Practical Applications**: Like other studies of DNN, the ultimate goal of training-free NAS is for real-world applications. For this reason, the reliability, flexibility, and robustness are needed to be considered in the design of a training-free NAS algorithm.

7. Conclusion

In this paper, we have reviewed the studies on training-free NAS for DNNs. The research in this area has flourished, indicating that training-free NAS is becoming a popular research topic in the field of DNNs. Table 3 provides an overall summarization of training-free NAS methods we discussed in this paper which consist of types of score functions (model-dependent vs. model-independent), search algorithms (from the random search, reinforcement, to metaheuristic algorithm), benchmarks, and datasets that are used in these studies. It can be easily seen from the table the functionalities and possibilities of these training-free NASs for different types of datasets and applications. The training-free NAS algorithm can be regarded as an unsupervised/semi-unsupervised approach because it does not rely on labeled data or explicit supervision during the search process in most cases The main reasons are: (1) The training-free NAS leverages the technique to estimate the performance of architectures without the need for individual architecture training and evaluation. (2) The obtained final optimal model may need to be trained to confirm the performance of the architecture or for subsequent applications. If the final model without training is used, the whole process will become an unsupervised approach; however, if the final model is trained with labeled data, the whole process will become a semi-unsupervised approach. It is important to note that the overall search process of training-free NAS remains unsupervised in nature. That is why training-free NAS methods offer the advantages of reduced computational resources and faster exploration of neural architectures. However, there is a trade-off to consider in terms of the accuracy and performance achieved by these methods compared to traditional NAS approaches. The use of training-free score functions introduces approximation, which may impact the evaluations and potentially lead to suboptimal architectures. According to our observations, striking the right balance between efficiency and effectiveness is crucial, and future research should focus on improving the accuracy of score functions and exploring efficient search algorithms to ensure that high-performing neural architectures are selected. Additionally, we have highlighted future research directions to enhance the development and application of training-free NAS algorithms. These include addressing computation cost and storage size, improving the performance of search algorithms, exploring decentralized computations, and considering practical applications. By addressing these challenges and future research directions, we can advance the field of training-free NAS and pave the way for more efficient systems that operate intelligently without significant human intervention. This paper serves as a call to action for researchers from different disciplines to contribute to the development of training-free NAS algorithms for DNNs.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors would like to thank the editor and anonymous reviewers for their valuable comments and suggestions on the paper. This work was supported in part by the National Science and Technology Council (NSTC) of Taiwan, R.O.C., under Contracts NSTC-111-2222-E-110-006-MY3, NSTC-112-2628-E-110-001-MY3, and NSTC-112-2634-F-110-001-MBK. All authors have approved the manuscript that is enclosed.

References

[1] C. Zhang, P. Patras, H. Haddadi, Deep learning in mobile and wireless networking: A survey, IEEE Commun. Surv. Tutor. 21 (3) (2019) 2224–2287.

[2] S. Ghosh, N. Das, I. Das, U. Maulik, Understanding deep learning techniques for image segmentation, ACM Comput. Surv. 52 (4) (2019) 1–35.

[3] S. Zhang, L. Yao, A. Sun, Y. Tay, Deep learning based recommender system: A survey and new perspectives, ACM Comput. Surv. 52 (1) (2019) 1–38.

[4] S. Pouyanfar, S. Sadiq, Y. Yan, H. Tian, Y. Tao, M.P. Reyes, M.-L. Shyu, S.-C. Chen, S.S. Iyengar, A survey on deep learning: Algorithms, techniques, and applications, ACM Comput. Surv. 51 (5) (2018) 1–36.

[5] Y. Lecun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE 86 (11) (1998) 2278–2324.

[6] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet classification with deep convolutional neural networks, Commun. ACM 60 (6) (2017) 84–90.

[7] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2014, arXiv preprint arXiv:1409.1556.

[8] P.P. Ray, A review on TinyML: State-of-the-art and prospects, J. King Saud Univ. - Comput. Inf. Sci. 34 (4) (2022) 1595–1623.

[9] Y. Chen, B. Zheng, Z. Zhang, Q. Wang, C. Shen, Q. Zhang, Deep learning on mobile and embedded devices: State-of-the-art, challenges, and future directions, ACM Comput. Surv. 53 (4) (2020) 1–37.

[10] Z. Chang, S. Liu, X. Xiong, Z. Cai, G. Tu, A survey of recent advances in edge-computing-powered artificial intelligence of things, IEEE Internet Things J. 8 (18) (2021) 13849–13875.

[11] L. Dutta, S. Bharali, TinyML meets IoT: A comprehensive survey, Int. Things 16 (2021) 100461.

[12] S. Soro, TinyML for ubiquitous edge AI, 2021, arXiv preprint arXiv:2102.01255.

[13] R. Sanchez-Iborra, A.F. Skarmeta, TinyML-enabled frugal smart objects: Challenges and opportunities, IEEE Circuits Syst. Mag. 20 (3) (2020) 4–18.

[14] T. Liang, J. Glossner, L. Wang, S. Shi, X. Zhang, Pruning and quantization for deep neural network acceleration: A survey, Neurocomputing 461 (2021) 370–403.

[15] L. Deng, G. Li, S. Han, L. Shi, Y. Xie, Model compression and hardware acceleration for neural networks: A comprehensive survey, Proc. IEEE 108 (4) (2020) 485–532.

[16] Q. Yao, M. Wang, H.J. Escalante, I. Guyon, Y. Hu, Y. Li, W. Tu, Q. Yang, Y. Yu, Taking human out of learning applications: A survey on automated machine learning, 2018, arXiv preprint arXiv:1810.13306.

[17] S. Fong, S. Deb, X.-S. Yang, How meta-heuristic algorithms contribute to deep learning in the hype of big data analytics, in: Proceedings of the Progress in Intelligent Computing Techniques: Theory, Practice, and Applications, Vol. 518, 2018, pp. 3–25.

[18] X. He, K. Zhao, X. Chu, AutoML: A survey of the state-of-the-art, Knowl.-Based Syst. 212 (2021) 106622.

[19] M.A. Amirabadi, M.H. Kahaei, S.A. Nezamalhosseini, Novel suboptimal approaches for hyperparameter tuning of deep neural network [under the shelf of optical communication], Phys. Commun. 41 (2020) 101057.

[20] P. Ren, Y. Xiao, X. Chang, P.-y. Huang, Z. Li, X. Chen, X. Wang, A comprehensive survey of neural architecture search: Challenges and solutions, ACM Comput. Surv. 54 (4) (2021) 1–34.

[21] T. Elsken, J.H. Metzen, F. Hutter, Neural architecture search: A survey, J. Mach. Learn. Res. 20 (1) (2019) 1997–2017.

[22] C.-W. Tsai, J.J.P.C. Rodrigues, Metaheuristic scheduling for cloud: A survey, IEEE Syst. J. 8 (1) (2014) 279–291.

[23] C. Liu, L.-C. Chen, F. Schroff, H. Adam, W. Hua, A.L. Yuille, L. Fei-Fei, Auto-DeepLab: Hierarchical neural architecture search for semantic image segmentation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 82–92.

[24] N. Wang, Y. Gao, H. Chen, P. Wang, Z. Tian, C. Shen, Y. Zhang, NAS-FCOS: Fast neural architecture search for object detection, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 11943–11951.

[25] Y. Gou, B. Li, Z. Liu, S. Yang, X. Peng, CLEARER: Multi-scale neural architecture search for image restoration, in: Proceedings of the Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 17129–17140.

[26] S. Ding, T. Chen, X. Gong, W. Zha, Z. Wang, Autospeech: Neural architecture search for speaker recognition, 2020, arXiv preprint arXiv:2005.03215.

[27] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, Commun. ACM 60 (6) (2017) 84–90.

[28] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2014, arXiv preprint arXiv:1409.1556.

[29] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 1–9.

[30] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.

[31] F.N. Iandola, S. Han, M.W. Moskewicz, K. Ashraf, W.J. Dally, K. Keutzer, SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and< 0.5 MB model size, 2016, arXiv preprint arXiv:1602.07360.

[32] B. Zoph, Q.V. Le, Neural architecture search with reinforcement learning, 2016, arXiv preprint arXiv:1611.01578.

[33] E. Real, A. Aggarwal, Y. Huang, Q.V. Le, Regularized evolution for image classifier architecture search, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2019, pp. 4780–4789.

[34] B. Zoph, V. Vasudevan, J. Shlens, Q.V. Le, Learning transferable architectures for scalable image recognition, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 8697–8710.

[35] B. Na, J. Mok, H. Choe, S. Yoon, Accelerating neural architecture search via proxy data, 2021, arXiv preprint arXiv:2106.04784.

[36] E. Real, A. Aggarwal, Y. Huang, Q.V. Le, Regularized evolution for image classifier architecture search, Proc. AAAI Conf. Artif. Intell. 33 (01) (2019) 4780–4789.

[37] P. Chrabaszcz, I. Loshchilov, F. Hutter, A downsampled variant of imagenet as an alternative to the cifar datasets, 2017, arXiv preprint arXiv:1707.08819.

[38] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, ImageNet: A large-scale hierarchical image database, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2009, pp. 248–255.

[39] T. Domhan, J.T. Springenberg, F. Hutter, Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves, in: Proceedings of the International Joint Conference on Artificial Intelligence, 2015, pp. 3460–3468.

[40] A. Klein, S. Falkner, J.T. Springenberg, F. Hutter, Learning curve prediction with Bayesian neural networks, in: Proceedings of the International Conference on Learning Representations, 2017.

[41] B. Baker, O. Gupta, R. Raskar, N. Naik, Accelerating neural architecture search using performance prediction, 2017, arXiv preprint arXiv:1705.10823.

[42] C. Liu, B. Zoph, M. Neumann, J. Shlens, W. Hua, L.-J. Li, L. Fei-Fei, A. Yuille, J. Huang, K. Murphy, Progressive neural architecture search, in: Proceedings of the European Conference on Computer Vision (ECCV), 2018, pp. 19–34.

[43] T. Chen, I. Goodfellow, J. Shlens, Net2net: Accelerating learning via knowledge transfer, 2015, arXiv preprint arXiv:1511.05641.
[44] H. Cai, T. Chen, W. Zhang, Y. Yu, J. Wang, Efficient architecture search by network transformation, Proc. AAAI Conf. Artif. Intell. 32 (1) (2018) 2787–2794.
[45] T. Elsken, J.-H. Metzen, F. Hutter, Simple and efficient architecture search for convolutional neural networks, 2017, arXiv preprint arXiv:1711.04528.
[46] H. Pham, M. Guan, B. Zoph, Q. Le, J. Dean, Efficient neural architecture search via parameters sharing, in: Proceedings of the International Conference on Machine Learning, 2018, pp. 4095–4104.
[47] H. Liu, K. Simonyan, Y. Yang, DARTS: Differentiable architecture search, in: International Conference on Learning Representations, 2019.
[48] C. White, M. Safari, R. Sukthanker, B. Ru, T. Elsken, A. Zela, D. Dey, F. Hutter, Neural architecture search: Insights from 1000 papers, 2023, arXiv preprint arXiv:2301.08727.
[49] X. Xie, X. Song, Z. Lv, G.G. Yen, W. Ding, Y. Sun, Efficient evaluation methods for neural architecture search: A survey, 2023, arXiv preprint arXiv:2301.05919.
[50] J. Mellor, J. Turner, A. Storkey, E.J. Crowley, Neural architecture search without training, in: Proceedings of the International Conference on Machine Learning, 2021, pp. 7588–7598.
[51] M. Javaheripi, S. Shah, S. Mukherjee, T.L. Religa, C.C.T. Mendes, G.H. de Rosa, S. Bubeck, F. Koushanfar, D. Dey, LiteTransformerSearch: Training-free on-device search for efficient autoregressive language models, in: Proceedings of the Conference on Automated Machine Learning, 2022.
[52] M.-T. Wu, H.-I. Lin, C.-W. Tsai, A training-free genetic neural architecture search, in: Proceedings of the ACM International Conference on Intelligent Computing and Its Emerging Applications, 2021, pp. 65–70.
[53] M.S. Abdelfattah, A. Mehrotra, Ł. Dudziak, N.D. Lane, Zero-cost proxies for lightweight NAS, in: Proceedings of the International Conference on Learning Representations, 2021.
[54] L. Xiang, Ł. Dudziak, M.S. Abdelfattah, T. Chau, N.D. Lane, H. Wen, Zero-cost proxies meet differentiable architecture search, 2021, arXiv preprint arXiv:2106.06799.
[55] N. Cavagnero, L. Robbiano, B. Caputo, G. Averta, FreeREA: Training-free evolution-based architecture search, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, 2023, pp. 1493–1502.
[56] N. Mokhtari, A. Nédélec, M. Gilles, P. De Loor, Improving neural architecture search by mixing a FireFly algorithm with a training free evaluation, in: Proceedings of the International Joint Conference on Neural Networks, 2022, pp. 1–8.
[57] Z. Zhang, Z. Jia, GradSign: Model performance inference with theoretical insights, in: Proceedings of the International Conference on Learning Representations, 2022.
[58] C. White, M. Khodak, R. Tu, S. Shah, S. Bubeck, D. Dey, A deeper look at zero-cost proxies for lightweight NAS, https://iclr-blog-track.github.io/2022/03/25/zero-cost-proxies/.
[59] A. Camero, J. Toutouh, E. Alba, Low-cost recurrent neural network expected performance evaluation, 2018, arXiv preprint arXiv:1805.07159.
[60] A. Camero, H. Wang, E. Alba, T. Bäck, Bayesian neural architecture search using a training-free performance metric, Appl. Soft Comput. 106 (2021) 107356.
[61] A. Camero, J. Toutouh, E. Alba, Reliable and fast recurrent neural network architecture optimization, 2021, arXiv preprint arXiv:2106.15295.
[62] V. Lopes, S. Alirezazadeh, L.A. Alexandre, EPE-NAS: Efficient performance estimation without training for neural architecture search, in: Proceedings of the International Conference on Artificial Neural Networks, 2021, pp. 552–563.
[63] W. Chen, X. Gong, Z. Wang, Neural architecture search on imagenet in four GPU hours: A theoretically inspired perspective, 2021, arXiv preprint arXiv:2102.11535.
[64] T. Do, N.H. Luong, Training-free multi-objective evolutionary neural architecture search via neural tangent kernel and number of linear regions, in: Proceedings of the International Conference on Neural Information Processing, 2021, pp. 335–347.
[65] B. Hanin, D. Rolnick, Complexity of linear regions in deep networks, in: Proceedings of the International Conference on Machine Learning, 2019, pp. 2596–2604.
[66] M. Lin, P. Wang, Z. Sun, H. Chen, X. Sun, Q. Qian, H. Li, R. Jin, Zen-NAS: A zero-shot NAS for high-performance image recognition, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 347–356.
[67] Z. Sun, M. Lin, X. Sun, Z. Tan, H. Li, R. Jin, MAE-DET: Revisiting maximum entropy principle in zero-shot NAS for efficient object detection, in: Proceedings of the International Conference on Machine Learning, 2022, pp. 20810–20826.
[68] W. Chen, W. Huang, X. Du, X. Song, Z. Wang, D. Zhou, Auto-scaling vision transformers without training, 2022, arXiv preprint arXiv:2202.11921.
[69] L. Theis, I. Korshunova, A. Tejani, F. Huszár, Faster gaze prediction with dense networks and fisher pruning, 2018, arXiv preprint arXiv:1801.05787.
[70] J. Turner, E.J. Crowley, M. O'Boyle, A. Storkey, G. Gray, Block-Swap: Fisher-guided block substitution for network compression on a budget, 2019, arXiv preprint arXiv:1906.04113.
[71] N. Lee, T. Ajanthan, P.H. Torr, SNIP: Single-shot network pruning based on connection sensitivity, 2018, arXiv preprint arXiv:1810.02340.
[72] C. Wang, G. Zhang, R. Grosse, Picking winning tickets before training by preserving gradient flow, 2020, arXiv preprint arXiv:2002.07376.
[73] H. Tanaka, D. Kunin, D.L. Yamins, S. Ganguli, Pruning neural networks without any data by iteratively conserving synaptic flow 33 (2020) 6377–6389.
[74] J. Lee, Y. Bahri, R. Novak, S.S. Schoenholz, J. Pennington, J. Sohl-Dickstein, Deep neural networks as gaussian processes, 2017, arXiv preprint arXiv:1711.00165.
[75] D.S. Park, J. Lee, D. Peng, Y. Cao, J. Sohl-Dickstein, Towards nngp-guided neural architecture search, 2020, arXiv preprint arXiv:2011.06006.
[76] A. Jacot, F. Gabriel, C. Hongler, Neural tangent kernel: Convergence and generalization in neural networks, in: Proceedings of the Conference on Neural Information Processing Systems, Vol. 31, 2018, pp. 8580–8589.
[77] H. Wang, Y. Wang, R. Sun, B. Li, Global convergence of MAML and theory-inspired neural architecture search for few-shot learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 9797–9808.
[78] Y. Shu, S. Cai, Z. Dai, B.C. Ooi, B.K.H. Low, NASI: Label-and data-agnostic neural architecture search at initialization, in: Proceedings of the International Conference on Learning Representations, 2022.
[79] J. Xu, L. Zhao, J. Lin, R. Gao, X. Sun, H. Yang, KNAS: Green neural architecture search, in: Proceedings of the International Conference on Machine Learning, 2021, pp. 11613–11625.
[80] M.-T. Wu, H.-I. Lin, C.-W. Tsai, A training-free neural architecture search algorithm based on search economics, IEEE Trans. Evol. Comput. (2023) 1.
[81] W. Chen, X. Gong, Y. Wei, H. Shi, Z. Yan, Y. Yang, Z. Wang, Understanding and accelerating neural architecture search with training-free and theory-grounded metrics, 2021, arXiv preprint arXiv:2108.11939.
[82] L.-T. Tran, M.S. Ali, S.-H. Bae, A feature fusion based indicator for training-free neural architecture search, IEEE Access 9 (2021) 133914–133923.
[83] C.W. Tsai, M.C. Chiang, Handbook of Metaheuristic Algorithms: From Fundamental Theories To Advanced Applications, Elsevier, 2023.
[84] F. Glover, Future paths for integer programming and links to artificial intelligence, Comput. Oper. Res. 13 (5) (1986) 533–549.

[85] C. Blum, A. Roli, Metaheuristics in combinatorial optimization: Overview and conceptual comparison, ACM Comput. Surv. 35 (3) (2003) 268–308.
[86] F. Glover, G.A. Kochenberger (Eds.), Handbook of Metaheuristics, Kluwer Academic Publishers, Boston, MA, 2003.
[87] D. Whitley, T. Starkweather, C. Bogart, Genetic algorithms and neural networks: Optimizing connections and connectivity, Parallel Comput. 14 (3) (1990) 347–361.
[88] D.L. Tong, R. Mintram, Genetic Algorithm-Neural Network (GANN): A study of neural network activation functions and depth of genetic algorithm search applied to feature selection, Int. J. Mach. Learn. Cybern. 1 (2010) 75–87.
[89] S. Ding, C. Su, J. Yu, An optimizing BP neural network algorithm based on genetic algorithm, Artif. Intell. Rev. 36 (2011) 153–162.
[90] X. He, K. Zhao, X. Chu, AutoML: A survey of the state-of-the-art, 2019, arXiv preprint arXiv:1908.00709.
[91] H. Rushing, J. Wisnowski, A. Karl, Design and Analysis of Experiments By Douglas Montgomery: A Supplement for using JMP, SAS Institute Inc., USA, 2013.
[92] J.S. Bergstra, R. Bardenet, Y. Bengio, B. Kégl, Algorithms for hyper-parameter optimization, in: Proceedings of the International Conference on Neural Information Processing, 2011, pp. 2546–2554.
[93] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, J. Mach. Learn. Res. 13 (2012) 281–305.
[94] Y. Ozaki, M. Yano, M. Onishi, Effective hyperparameter optimization using Nelder-Mead method in deep learning, IPSJ Trans. Comput. Vis. Appl. 9 (20) (2017) 1–12.
[95] J. Wu, X.-Y. Chen, H. Zhang, L.-D. Xiong, H. Lei, S.-H. Deng, Hyperparameter optimization for machine learning models based on Bayesian optimizationb, J. Electron. Sci. Technol. 17 (1) (2019) 26–40.
[96] B. Qolomany, M. Maabreh, A. Al-Fuqaha, A. Gupta, D. Benhaddou, Parameters optimization of deep learning models using particle swarm optimization, in: Proceedings of the International Wireless Communications and Mobile Computing Conference, 2017, pp. 1285–1290.
[97] T. Serizawa, H. Fujita, Optimization of convolutional neural network using the linearly decreasing weight particle swarm optimization, 2020, arXiv preprint arXiv:2001.05670.
[98] T. Elsken, J.H. Metzen, F. Hutter, Neural architecture search: A survey, J. Mach. Learn. Res. 20 (55) (2019) 1–21.
[99] M. Lin, R. Ji, Y. Zhang, B. Zhang, Y. Wu, Y. Tian, Channel pruning via automatic structure search, in: Proceedings of the International Joint Conference on Artificial Intelligence, 2020, pp. 673–679.
[100] Y. Zhou, G.G. Yen, Z. Yi, A knee-guided evolutionary algorithm for compressing deep neural networks, IEEE Trans. Cybern. 51 (3) (2019) 1626–1638.
[101] L. Xie, A. Yuille, Genetic CNN, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 1379–1388.
[102] P.R. Lorenzo, J. Nalepa, Memetic evolution of deep neural networks, in: Proceedings of the Genetic and Evolutionary Computation Conference, 2018, pp. 505–512.
[103] Y. Sun, B. Xue, M. Zhang, G.G. Yen, Evolving deep convolutional neural networks for image classification, IEEE Trans. Evol. Comput. 24 (2) (2020) 394–407.
[104] Y. Sun, B. Xue, M. Zhang, G.G. Yen, Completely automated CNN architecture design based on blocks, IEEE Trans. Neural Netw. Learn. Syst. 31 (4) (2020) 1242–1254.
[105] L.T. Tran, S.-H. Bae, Training-free hardware-aware neural architecture search with reinforcement learning, J. Broadcast Eng. 26 (7) (2021) 855–861.
[106] M. Zhang, S. Su, S. Pan, X. Chang, W. Huang, G. Haffari, Differentiable architecture search without training nor labels: A pruning perspective, 2021, arXiv preprint arXiv:2106.11542.
[107] S.E. Arman, S.A. Deowan, IGWO-SS: Improved grey wolf optimization based on synaptic saliency for fast neural architecture search in computer vision, IEEE Access 10 (2022) 67851–67869.
[108] R. Geada, A.S. McGough, SpiderNet: Hybrid differentiable-evolutionary architecture search via train-free metrics, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 2022, pp. 1962–1970.
[109] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, F. Hutter, NAS-Bench-101: Towards reproducible neural architecture search, in: Proceedings of the International Conference on Machine Learning, 2019, pp. 7105–7114.
[110] X. Dong, Y. Yang, NAS-Bench-201: Extending the scope of reproducible neural architecture search, in: Proceedings of the International Conference on Learning Representations, 2020.
[111] X. Dong, L. Liu, K. Musial, B. Gabrys, NATS-Bench: Benchmarking NAS algorithms for architecture topology and size, IEEE Trans. Pattern Anal. Mach. Intell. 44 (7) (2022) 3634–3646.
[112] J. Siems, L. Zimmer, A. Zela, J. Lukasik, M. Keuper, F. Hutter, NAS-Bench-301 and the case for surrogate benchmarks for neural architecture search, 2020, arXiv preprint arXiv:2008.09777.
[113] A. Zela, J. Siems, F. Hutter, NAS-Bench-1Shot1: Benchmarking and dissecting one-shot neural architecture search, 2020, arXiv preprint arXiv:2001.10422.
[114] T. Mikolov, M. Karafiát, L. Burget, J. Cernockỳ, S. Khudanpur, Recurrent neural network based language model, in: Proceedings of the Annual Conference of the International Speech Communication Association, Vol. 2, 2010, pp. 1045–1048, no. 3.
[115] S. Merity, C. Xiong, J. Bradbury, R. Socher, Pointer sentinel mixture models, 2016, arXiv preprint arXiv:1609.07843.
[116] N. Klyuchnikov, I. Trofimov, E. Artemova, M. Salnikov, M. Fedorov, A. Filippov, E. Burnaev, NAS-Bench-NLP: Neural architecture search benchmark for natural language processing, IEEE Access 10 (2022) 45736–45747.
[117] A.R. Zamir, A. Sax, W. Shen, L.J. Guibas, J. Malik, S. Savarese, Taskonomy: Disentangling task transfer learning, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 3712–3722.
[118] Y. Duan, X. Chen, H. Xu, Z. Chen, X. Liang, T. Zhang, Z. Li, TransNAS-Bench-101: Improving transferability and generalizability of cross-task neural architecture search, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 5251–5260.
[119] G.D. Clifford, C. Liu, B. Moody, H.L. Li-wei, I. Silva, Q. Li, A. Johnson, R.G. Mark, AF classification from a short single lead ECG recording: The PhysioNet/computing in cardiology challenge 2017, in: Proceedings of the Computing in Cardiology, 2017, pp. 1–4.
[120] E. Feingold, L. Pachter, The ENCODE (ENCyclopedia of DNA elements) project, Science 306 (5696) (2004) 636–640.
[121] R. Tu, M. Khodak, N. Roberts, A. Talwalkar, NAS-Bench-360: Benchmarking diverse tasks for neural architecture search, 2021, arXiv preprint arXiv:2110.05668.
[122] H. Chen, M. Lin, X. Sun, H. Li, NAS-bench-zero: A large scale dataset for understanding zero-shot neural architecture search, 2022, Openreview preprint [Online]. Available: https://openreview.net/forum?id=hP-SILoczR.
[123] A. Krizhevsky, Learning Multiple Layers of Features from Tiny Images, Tech. Rep., University of Toronto, 2009.
[124] T.S. Cohen, M. Geiger, J. Köhler, M. Welling, Spherical CNNs, 2018, arXiv preprint arXiv:1801.10130.
[125] M. Atzori, A. Gijsberts, S. Heynen, A.-G.M. Hager, O. Deriaz, P. Van Der Smagt, C. Castellini, B. Caputo, H. Müller, Building the Ninapro database: A resource for the biorobotics community, in: Proceedings of the IEEE RAS & EMBS International Conference on Biomedical Robotics and Biomechatronics, 2012, pp. 1258–1265.
[126] K. Zhang, J.S. Bloom, deepCR: Cosmic ray rejection with deep learning, Astrophys. J. 889 (1) (2020) 24.
[127] F. Petitjean, J. Inglada, P. Gançarski, Satellite image time series analysis under time warping, IEEE Trans. Geosci. Remote Sens. 50 (8) (2012) 3081–3095.

[128] X. Ning, C. Tang, W. Li, Z. Zhou, S. Liang, H. Yang, Y. Wang, Evaluating efficient performance estimators of neural architectures, in: Proceedings of the Neural Information Processing Systems, Vol. 34, 2021, pp. 12265–12277.
[129] A. Krishnakumar, C. White, A. Zela, R. Tu, M. Safari, F. Hutter, NAS-bench-suite-zero: Accelerating research on zero cost proxies, in: Proceedings of the Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2022.
[130] H. Pham, M. Guan, B. Zoph, Q. Le, J. Dean, Efficient neural architecture search via parameters sharing, in: Proceedings of the International Conference on Machine Learning, 2018, pp. 4095–4104.
[131] H. Liu, K. Simonyan, Y. Yang, DARTS: Differentiable architecture search, in: Proceedings of the International Conference on Learning Representations, 2019.
[132] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, 2018, arXiv preprint arXiv:1810.04805.
[133] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: Proceedings of the Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 1877–1901.
[134] D.H. Stolfi, E. Alba, X. Yao, Predicting car park occupancy rates in smart cities, in: Proceedings of the International Conference on Smart Cities, 2017, pp. 107–117.
[135] L.M. Candanedo, V. Feldheim, D. Deramaix, Data driven prediction models of energy use of appliances in a low-energy house, Energy Build. 140 (2017) 81–97.
[136] F.J.F. Urbano, E.A. Torres, BIN-CT: Sistema inteligente para la gestión de la recogida de residuos urbanos, in: Proceedings of the International Greencities Congress, 2018, pp. 187–198.
[137] B.-J. Chen, M.-W. Chang, et al., Load forecasting using support vector machines: A study on EUNITE competition 2001, IEEE Trans. Power Syst. 19 (4) (2004) 1821–1830.
[138] I. Radosavovic, J. Johnson, S. Xie, W.-Y. Lo, P. Dollár, On network design spaces for visual recognition, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 1882–1890.
[139] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C.L. Zitnick, Microsoft COCO: Common objects in context, in: Proceedings of the European Conference on Computer Vision, 2014, pp. 740–755.
[140] C. Chelba, T. Mikolov, M. Schuster, Q. Ge, T. Brants, P. Koehn, T. Robinson, One billion word benchmark for measuring progress in statistical language modeling, 2013, arXiv preprint arXiv:1312.3005.
[141] A. Mehrotra, A.G.C. Ramos, S. Bhattacharya, Ł. Dudziak, R. Vipperla, T. Chau, M.S. Abdelfattah, S. Ishtiaq, N.D. Lane, NAS-Bench-ASR: Reproducible neural architecture search for speech recognition, in: Proceedings of the International Conference on Learning Representations, 2020.
[142] J.S. Garofolo, L.F. Lamel, W.M. Fisher, J.G. Fiscus, D.S. Pallett, DARPA TIMIT Acoustic-Phonetic Continous Speech Corpus CD-ROM. NIST Speech Disc 1-1.1, Vol. 93, 1993.
[143] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, Q.V. Le, MnasNet: Platform-aware neural architecture search for mobile, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 2820–2828.
[144] D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, R. Garnett, Matching networks for one shot learning, in: Proceedings of the Neural Information Processing Systems, Vol. 29, 2016, pp. 3637–3645.
[145] M. Ren, E. Triantafillou, S. Ravi, J. Snell, K. Swersky, J.B. Tenenbaum, H. Larochelle, R.S. Zemel, Meta-learning for semi-supervised few-shot classification, 2018, arXiv preprint arXiv:1803.00676.
[146] J. Lin, W.-M. Chen, J. Cohn, C. Gan, S. Han, MCUNet: Tiny deep learning on IoT devices, in: Proceedings of the Conference on Neural Information Processing Systems, 2020, pp. 11711–11722.
[147] J. Lin, W.-M. Chen, H. Cai, C. Gan, S. Han, MCUNetV2: Memory-efficient patch-based inference for tiny deep learning, in: Proceedings of the Conference on Neural Information Processing Systems, 2021.